Bryant Park Consulting

CONFIDENTIAL · JUNE 2026 · V2026-06-23.1112

NSPB Cube Optimization Review

Northwind · Plan cube size, data distribution & calc-performance analysis, with a prioritized cleanup plan.

581 GB
customer data on disk

36 of 47
scenarios are stale copies

235 min
slowest calc (CConv_Plan)

-31%
of input blocks clearable

bryantparkconsulting.com

NetSuite Planning & Budgeting

NSPB Cube Optimization Report — Northwind

Prepared by BPC · source: LCM metadata + level-0 data export + Activity Report · ***v2026-06-23.1112***

Executive summary

- ⚠ **Application size: 581 GB** customer data on disk (264 GB Essbase + 265 GB snapshots). Plan cube page file alone = **177,418 MB (173 GB)**.
- ⚠ **Plan cube: 3,645,406 input (level-0) blocks** expand to 337,653,497 stored blocks via full aggregation (~93× — applied to all 47 scenarios and 12 years, including stale ones).
- ✗ **Slowest calc: CConv_Plan** ran **235:00 min** (3.9 h) — by far the #1 calc-time hotspot. Avg calc execution 111s.
- ✗ **Scenario bloat: 36 stale / one-off scenarios** hold 29.5% of the input data — the lowest-risk size & calc-time reduction is to clear them.
- 💡 **Two biggest levers:** (1) **stop aggregating stale data** — clear the stale scenarios and scope the AGG rules to active scenarios/years (Part 1); and (2) **fix the runaway currency-conversion calc** (Part 2).

Everything in this report is presented as ***suggested changes*** *based on the LCM, level-0 export and Activity Report. Each item should be reviewed together with the team that owns the application and validated in a test environment before applying — usage patterns we cannot see (reporting needs, close-process dependencies) may change a recommendation.*

How to read this report: ✓ working well / keep as-is · ⚠ caution — verify before relying on it · ✗ issue found in the data · 💡 suggested change to validate. Best-practice references follow Oracle EPM Cloud / Hybrid BSO guidance (sources in the footer).

Health check at a glance

Working well

- ✓ **Hybrid mode is enabled**
  ready to push aggregations to dynamic calc
- ✓ **Clustering ratio 99%**
  data is well-ordered on disk
- ✓ **Dense/sparse design is appropriate**
  Account × Period — no re-architecting needed
- ✓ **84% of data in active years**
  FY25–FY26 — current and relevant
- ✓ **UI is responsive**
  avg session, only a small % of requests over 2s

Suggested changes

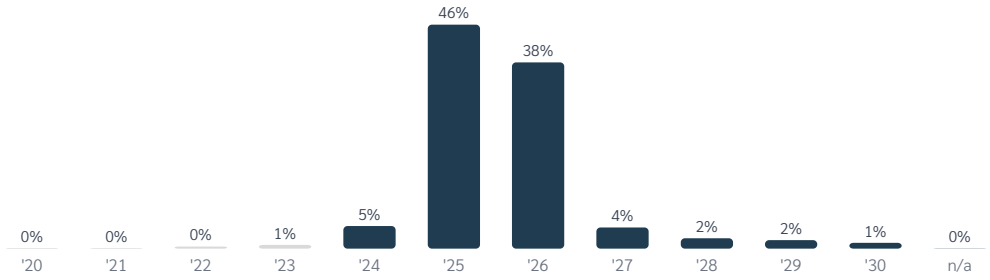
- ✗ **Plan page file 173 GB (581 GB on disk)**
  stale scenarios are aggregated too — clearing them cuts it
- ✗ **Aggregations stored for all 47 scenarios (~93x per input block)**
  clear stale scenarios + scope AGG rules to active data
- ✗ **CConv_Plan calc runs 235:00 min (3.9 h)**
  runaway — scope the FIX, run incrementally
- ✗ **ADMIN Datacopy avg 11:04 (4×/day)**
  high-cost repeated copy — tighten scope
- ✗ **36 stale/one-off scenarios = 29.5% of input blocks**
  clear/archive to shrink size + calc time
- ✗ **Outline warnings present**
  dynamic-calc L0 without formula; shared-descendant members

Part 1 — Space: where the data lives and what can be freed

In plain terms: this part shows which years and scenarios hold the data, which of them are stale copies that can be deleted, and how much space that would free.

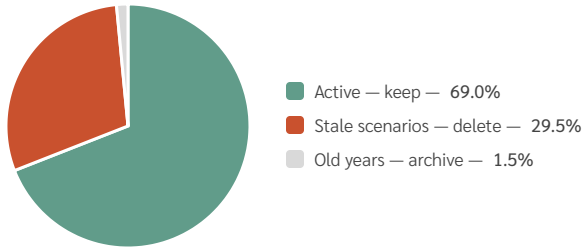
1.1 Where the data actually is

Blocks by year (grey = archive candidate)



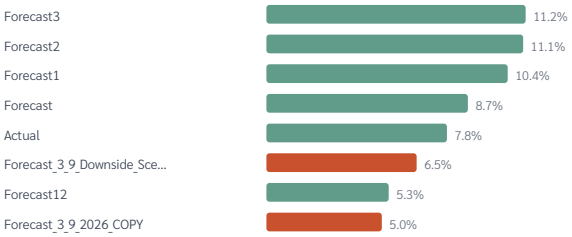
Each bar = how much of the typed/loaded data sits in that fiscal year. Almost everything lives in FY25–FY26; the grey years are history that can be archived.

What the input data is made of



Green = scenarios in active use (keep). Red = stale one-off copies (delete). Grey = old years (archive).

Top scenarios — keep vs delete candidate



Green = in active use. Red = delete candidates (dated copies, tests, downside runs). Full list in Section 1.2.

By year (% of input blocks):

- FY24: 4.7% (24,673)
- FY25: 46.1% (243,913)
- FY26: 38.3% (202,785)
- FY27: 4.4% (23,146)
- FY28: 2.1% (11,208)
- FY29: 1.7% (9,151)
- FY30: 1.2% (6,388)
- (years under 1% omitted — full distribution in the chart above)

By scenario (top 5 of 47 — full split in the chart above):

- Forecast3: 11.2% (59,358)
- Forecast2: 11.1% (58,792)
- Forecast1: 10.4% (55,246)
- Forecast: 8.7% (46,144)
- Actual: 7.8% (41,342)

1.2 Scenario × year — deletion candidates

A scenario is a **candidate** when its name is a one-off/dated/test copy **or** >90% of its data sits in archived years (FY20–FY23).

Scenario	Blocks	% of input blocks	Years with data	Why
Forecast_3_9_Downside_Scenario	34,385	6.5%	FY26,FY27,FY28,FY29,FY30	one-off/dated name
Forecast_3_9_2026_COPY	26,422	5.0%	FY26	one-off/dated name
Forecast_11_1_2025	20,981	4.0%	FY25,FY26,FY27,FY28,FY29	one-off/dated name
Test Forecast	19,913	3.8%	FY24,FY25,FY26,FY27,FY28,FY29	one-off/dated name
ClearScenariData	18,799	3.6%	FY25,FY26	one-off/dated name
Forecast_2024-2027_LRP_4_3_24	3,382	0.6%	FY24,FY25,FY26,FY27	one-off/dated name
Budget_2025_LRP_v1	2,978	0.6%	FY25,FY26,FY27	one-off/dated name
Forecast_12_12_24	1,885	0.4%	FY24	one-off/dated name
Forecast_10_2_2025	1,534	0.3%	FY25	one-off/dated name
Forecast_9_3_2025	1,512	0.3%	FY25	one-off/dated name
Forecast_6_6	1,492	0.3%	FY25	one-off/dated name
Forecast_8_4_2025	1,478	0.3%	FY25	one-off/dated name
Forecast_5_7	1,476	0.3%	FY25	one-off/dated name
Forecast_12_12_23	1,448	0.3%	FY23	one-off/dated name; old-years-only
2025_OP2_Budget_Transfers	1,422	0.3%	FY25,FY26	one-off/dated name
2025_OP+_Budget_Transfers	1,422	0.3%	FY25,FY26	one-off/dated name
Forecast_4_8	1,416	0.3%	FY25	one-off/dated name
Forecast_3_9	1,322	0.2%	FY25	one-off/dated name
Forecast_12_12_22	1,240	0.2%	FY22	one-off/dated name; old-years-only
Forecast_2_10	1,202	0.2%	FY25	one-off/dated name
Forecast_1_11	1,176	0.2%	FY25	one-off/dated name
Forecast_12_12_21	784	0.1%	FY21	one-off/dated name; old-years-only
Forecast_12_12_20	744	0.1%	FY20	one-off/dated name; old-years-only
Budget_2025_OP+_1_23	738	0.1%	FY25	one-off/dated name
Forecast_9_3_24	620	0.1%	FY24	one-off/dated name
Forecast_10_2_24	618	0.1%	FY24	one-off/dated name
Forecast_8_4_24	612	0.1%	FY24	one-off/dated name
Forecast_3_9_24	590	0.1%	FY24	one-off/dated name
Forecast_4_8_24	586	0.1%	FY24	one-off/dated name
Forecast_6_6_24	584	0.1%	FY24	one-off/dated name
Forecast_5_7_24	582	0.1%	FY24	one-off/dated name
Forecast_7_5_24	582	0.1%	FY24	one-off/dated name

Scenario	Blocks	% of input blocks	Years with data	Why
Forecast_2_10_24	580	0.1%	FY24	one-off/dated name
Forecast_1_11_24	558	0.1%	FY24	one-off/dated name
Forecast_11_1_24	552	0.1%	FY24	one-off/dated name
No Scenario	277	0.1%	FY23,FY24,FY25,FY26,No Year	one-off/dated name

Total candidates: 36 scenarios = 155,892 blocks = 29.5% of input (level-0) blocks.

1.3 Keep (active scenarios)

Forecast3 (11.2%) · Forecast2 (11.1%) · Forecast1 (10.4%) · Forecast (8.7%) · Actual (7.8%) · Forecast12 (5.3%) · Forecast7 (4.4%) · Forecast10 (4.1%) · Budget_2026_OP (3.6%) · Forecast4 (3.3%) · Budget_OP2 (0.7%)

1.4 Empty-block and zero-value bloat (to confirm)

❌ The level-0 data export contains **529,274 blocks with data**, while Essbase reports **3,645,406 level-0 blocks** — a 6.9× gap. A column export only writes blocks that hold values, so the difference (~3,116,132 blocks) is likely **#Missing-only blocks** left behind by CLEARDATA/copies.

💡 Suggested change: `CLEARBLOCK EMPTY` / explicit restructure (after a snapshot) — potentially a large, low-risk size win. Validate the count in test first.

Zero-valued cells — stored as if they were data

⚠️ The export was also scanned cell by cell: **1,580,715 of the 29,155,286 loaded cells (5.4%) hold a literal 0** instead of #Missing. Essbase stores, aggregates and currency-converts a 0 exactly like real data — these cells keep blocks alive and add calc work while carrying no information. Typical sources: data loads that send 0 instead of skipping the cell, and calcs/copies that write 0.

💡 Suggested change — a small **clear-zeros custom calc** run after the data loads, followed by `CLEARBLOCK EMPTY` so blocks that become fully empty are removed (illustrative — member lists to validate in test):

```
/* Clear-zeros — turn stored 0s back into #Missing (illustrative) */
SET UPDATECALC OFF;
FIX (@LEVMBRS("Account", 0), <active Scenario/Version/Years only>,
@LEVMBRS("Department", 0), @LEVMBRS("Location", 0), ...)
"TP1" (IF ("TP1" == 0) "TP1" = #Missing; ENDIF;)
/* …repeat for TP2..TP12 and BegBalance… */
ENDFIX

CLEARBLOCK EMPTY; /* drop blocks that are now all #Missing */
```

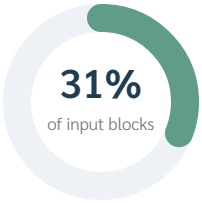
Scoped to the active scenarios this is low-risk; if any report intentionally distinguishes 0 from #Missing (rare), exclude those accounts.

1.5 Historical years & archival

- ✅ The cube carries **11 fiscal years** (FY20–FY30). **84.4%** sits in FY25–FY26; the working window used for cleanup decisions is FY24+ (Section 1.2).
- ⚠️ **FY20–FY23 actuals = 1.5%** of input blocks (7,822 blocks). Small in block terms, but they sit in the BSO plan cube unnecessarily.
- 💡 Keep ~2–3 years of actuals live for YoY; **archive FY20–FY22 to the ASO reporting cube (Rpt)** — it already exists (13 MB ASO) and is built for historical query. Then clear those years from the BSO Plan cube.
- 💡 Note: the **scenario** cleanup (Section 1.2) is a far bigger lever than the year archival — do both, scenarios first.

1.6 Estimated space impact — what the cleanup frees

Cleanup impact



529,274 input blocks
→ 373,382 after cleanup
–31%

Deleting the stale copies removes 31% of the input data — and their stored summaries go with them.

Plan page file — today vs after cleanup

Estimated, proportional: if the deleted scenarios are aggregated like the rest, the 173 GB page file drops to ~120 GB. Validate in a test environment.

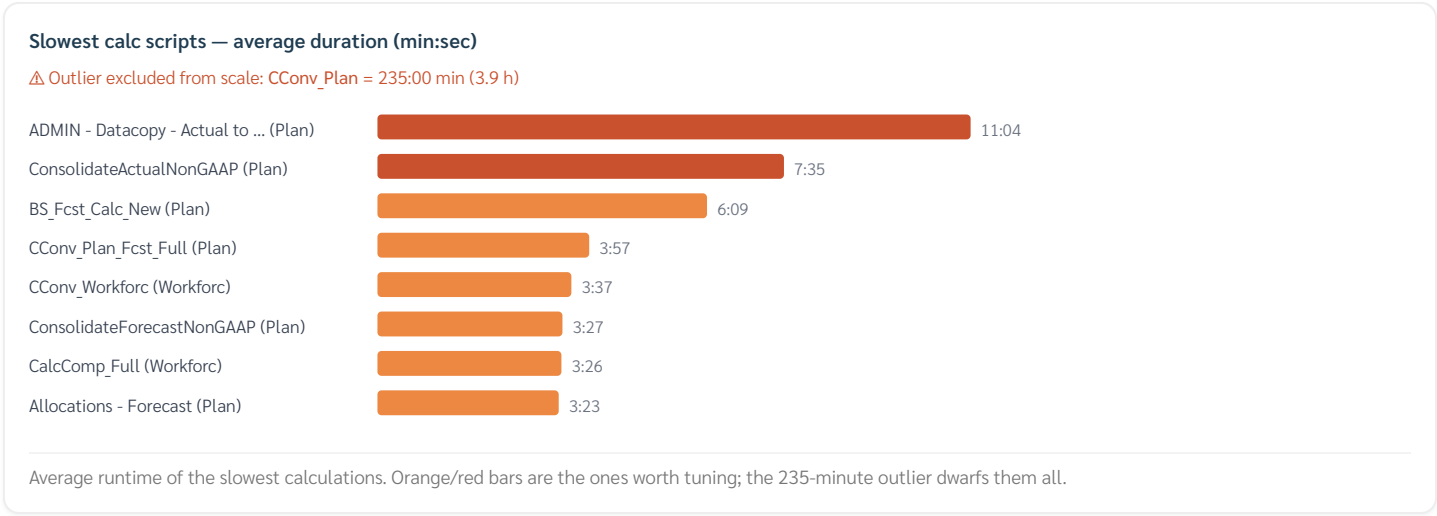
*How to read this: the percentages in 1.1–1.5 are measured on **input (level-0) blocks** — the cells users actually typed or loaded. But every input block also carries stored summary (upper-level) blocks created by aggregation. When an input block is deleted, its summaries disappear with it.*

- ✅ **Measured (exact):** stale scenarios + old years = **163,714 input blocks = 30.9% of input data**.
- 💡 **Estimated (proportional):** if those scenarios are aggregated like the rest of the cube, clearing them also removes ≈31% of the stored summary blocks — roughly **54 GB of the 173 GB Plan page file**. _This is an estimate: per-scenario aggregation varies (a test copy may never have been aggregated); validate in a test environment._
- 💡 On top of that, the **empty-block cleanup (1.4)** and not re-aggregating stale data (Part 2) compound the savings.

Part 2 — Calculation speed: what runs slow and why

In plain terms: this part shows which calculations take the longest, compares the scripts against Oracle's best practices, and explains — for the three slowest — how each works today and what would make it faster.

2.1 Slowest calculations (Activity Report)



- ✕ **Longest single run: CConv_Plan = 235:00 min** (3.9 h). This one currency-conversion calc dominates the whole day's calc time.

Top calc scripts by average duration:

Calc script	Cube	Execs	Avg	Max
CConv_Plan	Plan	1	235:00	235:00
ADMIN - Datacopy - Actual to Forecast	Plan	4	11:04	19:22
ConsolidateActualNonGAAP	Plan	1	7:35	7:35
BS_Fcst_Calc_New	Plan	1	6:09	6:09
CConv_Plan_Fcst_Full	Plan	21	3:57	4:12
CConv_Workforc	Workforc	10	3:37	3:43
ConsolidateForecastNonGAAP	Plan	1	3:27	3:27
CalcComp_Full	Workforc	10	3:26	3:36
Allocations - Forecast	Plan	3	3:23	3:36

Script-level findings and suggested rewrites for the top 3 are in **Section 2.3**.

Runtime metrics: Essbase requests 497 min/period · avg calc 111s · avg restructure 51s · max 12 calc threads.

2.2 Calc best-practice audit (Oracle EPM Cloud)

This is an **EPM Cloud Hybrid BSO** application (Essbase 21.7.4.0.0.024). All BSO calc best practices apply, plus Hybrid considerations. Of **189** business rules with scripts: **48 use parallel calc** (CALCPARALLEL/FIXPARALLEL), **141 do not**; 38 use CREATENONMISSINGBLK/CREATEBLOCKONEQ; 13 set AGGMISSG.

Best practice (Oracle EPM)	Tenant status	Action
Parallelize heavy aggregations/copies (CALCPARALLEL = cores−1, FIXPARALLEL for AGG/DATACOPY)	✕ Slowest rules without parallel: **CConv_Plan, CConv_Workforc, ConsolidateActualNonGAAP,**	💡 Add `SET CALCPARALLEL 11;` / FIXPARALLEL to **CConv_Plan** first (235:00

Best practice (Oracle EPM)	Tenant status	Action
	ConsolidateForecastNonGAAP	min, no parallel today)
Tight FIX scope; avoid converting/aggregating more than needed	❌ CConv_Plan FIXes the full Forecast scenario	💡 Scope to active years + run incrementally (last closed month forward)
Use CREATENONMISSINGBLK/CREATEBLOCKONEQ **sparingly, inside a tight FIX**	⚠️ 38 rules use it	💡 Audit those 38 rules — over-creation materializes empty blocks and inflates size
SET AGGMISSG matches version design (OFF for target/top-down, ON for bottom-up level-0)	⚠️ 13 rules set it	💡 Verify each matches its version type to avoid wrong #Missing aggregation
Order sparse aggregation **fewestmost blocks**; fix outline hourglass order	❌ 32 hourglass deviations on sparse	💡 Re-order aggregating sparse dims **smallestlargest** stored members (concrete order in Section 3.5); non-aggregating sparse last
Minimize dynamic-calc dependencies inside calcs; resolve outline warnings	⚠️ dynamic-calc L0 members without formula flagged	💡 Clean the outline warnings (Section 3.3)
Periodic **explicit restructure** to drop #Missing blocks & defrag	⚠️ implicit refreshes ~10:12 min	💡 Schedule a periodic full restructure, especially after cleanup
Set CALCPARALLEL ≤ available threads (12 max)	—	💡 Use 11 on heavy rules; never assume higher = faster — test

2.3 Deep dive — are the top 3 slowest calcs well-built?

For each, the actual LCM script reviewed: **how it works today vs. how it could be faster** (💡 = suggested change, to validate in test).

1. CConv_Plan — avg 235:00, max 235:00, 1 run/day (Activity Report window)

Converts financial data from CAD, AUD, and GBP to USD for a user-selected scenario. It applies different FX rates (Ending, Historical, Average) based on account properties and then aggregates the results. It also calculates the 'Load' account for specific equity members.

Area	Today	💡 Suggested change
Parallelism	runs **single-threaded** — one CPU for the whole script.	add `SET CALCPARALLEL 11;` (or wrap the heavy FIX/aggregation in FIXPARALLEL) to use all 12 threads.
Calc mode	sets `@CALCMODE(BOTTOMUP)` then `@CALCMODE(BLOCK)` — BLOCK **cancels** bottom-up and calculates every potential cell.	keep **BOTTOMUP** only — skips millions of empty cells on existing-data calcs.
Scope	references **18 year variables** (full history + forecast) every run.	build an **incremental** version (changed periods / last-closed-month forward) for routine runs; keep the full version for resets.
Aggregation	aggregates 7 sparse dimensions **sequentially** (`@IDESCENDANTS`).	order them **fewest-blocks-first** and use a single `AGG(...)` or FIXPARALLEL.
Block creation	uses `SET CREATENONMISSINGBLK ON`.	keep its FIX narrow so it does not materialize empty blocks.

Example — how the script looks today vs. the suggested shape. The left block is extracted from the actual LCM script; the right is illustrative and must be validated in test:

```
/* TODAY — extracts from the actual CConv_Plan script */
FIX ({CConv_Scenario}, &CurrentVersion)
FIX (@LEVMBRS("Account",0), @LEVMBRS("Period",0), ...all 9 dims at level 0)
"USD_Reporting" (
IF (@ISMBR(&BSOldestYr:&PriorYr) OR ...) ← year filter sits INSIDE
@CALCMODE(BOTTOMUP); @CALCMODE(BLOCK); the member block: every
...conversion logic... year's blocks still get
) visited; BLOCK cancels
ENDFIX BOTTOMUP
/* Aggregation — 7 sparse dims, sequential */
FIX (&BSOldestYr:&PriorYr, &LastClosedYr, &FcstYr1, &FcstYr2, ...)
@IDESCENDANTS("Subsidiary"); @IDESCENDANTS("Department");
@IDESCENDANTS("Class"); @IDESCENDANTS("Location");
@IDESCENDANTS("Relationship"); @IDESCENDANTS("Tracker"); @IDESCENDANTS("Item");
ENDFIX
ENDFIX
/* note: no SET CALCPARALLEL anywhere — the whole 3.9h run uses ONE thread */
```

```
/* SUGGESTED shape (illustrative — validate in test) */
SET CALCPARALLEL 11; ← use the 12 available threads
FIX ({CConv_Scenario}, &CurrentVersion,
&LastClosedYr, &FcstYr1, &FcstYr2) ← years in the FIX, not in IFs:
FIX (@LEVMBRS("Account",0), @LEVMBRS("Period",0), ...) blocks outside the active
"USD_Reporting" ( window are never touched
@CALCMODE(BOTTOMUP); ← drop @CALCMODE(BLOCK)
...same conversion logic...
)
ENDFIX
/* Aggregation — one parallel pass, smallest dimension first */
FIX ("USD_Reporting", @RELATIVE("Input Currencies",0))
AGG ("Class","Location","Item","Relationship","Subsidiary","Tracker","Department");
ENDFIX
ENDFIX
/* keep the full-history version (&BSOldestYr:&PriorYr) as a separate
on-demand rule for restatements — not in the routine run */
```

💡 **Suggested change — run it at night.** At 235:00 min it was launched **interactively** (blocking a user session for hours). Schedule the full run via **Application Jobs Schedule Jobs** (rule type, daily, e.g. 02:00 after the data loads) or an **EPM Automate** `runBusinessRule` step in the nightly pipeline — and give users a light incremental version for intraday.

2. ADMIN - Datacopy - Actual to Forecast — avg 11:04, max 19:22, 4 runs/day (Activity Report window)

Copies Actual data to the Forecast scenario for closed periods, including Income Statement, Balance Sheet, and FX rates. It also performs reclassifications, clears specific data, and aggregates the Income Statement.

Area	Today	💡 Suggested change
Scope	references **6 year variables** (full history + forecast) every run.	build an **incremental** version (changed periods / last-closed-month forward) for routine runs; keep the full version for resets.

💡 **Suggested change — reduce frequency.** It runs **4×/day** (~44:16 total). Confirm each run is needed, or trigger it only when its source data actually changes.

3. ConsolidateActualNonGAAP — avg 7:35, max 7:35, 1 run/day (Activity Report window)

Copies data from USD to USD_Reporting and from Actual to Forecast for specific accounts and periods, then aggregates data across multiple dimensions for Actual and Forecast scenarios.

Area	Today	💡 Suggested change
Parallelism	runs **single-threaded** — one CPU for the whole script.	add `SET CALCPARALLEL 11;` (or wrap the heavy FIX/aggregation in FIXPARALLEL) to use all 12 threads.
Data copy	does a broad `DATACOPY`/`CLEARDATA` single-threaded.	tighten the source/target FIX and run it under FIXPARALLEL.

2.4 Where the cube grows — blocks created

Which calcs CREATE the most blocks (this is what inflates the 173 GB page file):

Calc script	Blocks created	FIX scope
BS_Fcst_Calc_New	6,400	`FIXPARALLEL(4, Forecast, Base, @RELATIVE(Currency,0))`
ADMIN - Daily Sync AGG	1,580	`FIXPARALLEL(4, FY26, TP1:TP5, Base, Actual, @RELATIVE(Amount,0)...)`
ConsolidateForecastNonGAAP	1,280	`FIX(Forecast, FY26, FY27, FY28, Base, Load)`
ConsolidateActualNonGAAP	926	`FIX(Actual, Forecast, FY26, Base, Load)`
CConv_Workforc	128	`FIX(Forecast, Base, FY26, FY27) — runs hourly`

- **BS_Fcst_Calc_New** creates the most (6,400) via a broad FIXPARALLEL across all currencies — 💡 a prime candidate to scope tighter and/or dynamic-calc the targets.
- ❗ Several consolidations create blocks for **non-GAAP** across multiple years — confirm those upper levels need to be stored vs computed on the fly (Hybrid).

Part 3 — Cube design & data flow (reference)

In plain terms: this part checks whether the cube's structure (which dimensions are dense vs sparse, what is stored vs computed) is well designed — verdict: it is — and documents how data flows in from NetSuite.

3.1 Dimension profile (from LCM — all cubes combined)

*Note: the LCM lists members across ALL cubes. Employee belongs to the Workforc cube, Index to Details; Account is **dense** in Plan. Per-cube block analysis is in Sections 3.2–3.5.*

Dimension	Members	Dynamic-calc	Stored	Missing alias
Employee	4,999	4	4,995	0
Account	3,160	591	1,545	406
Index	3,115	0	3,115	1
Department	784	60	332	11
Location	116	3	86	6
Tracker	107	14	67	26
Scenario	92	22	57	35
Subsidiary	59	0	35	5
Relationship	46	0	32	5
Period	43	14	15	15

- ✅ **Plan cube's sparse block drivers:** Department (382 stored), Tracker (60 stored), Scenario (60 stored), Subsidiary (57 stored) — the product of stored sparse members bounds Plan's block count. (Employee drives Workforc; Index drives Details.)
- ⚠️ **Alias gaps:** Account (406) — these members display raw names in reports (quick win to fix).

3.2 Block design — dense vs sparse

- ✅ **Dense dimensions** (define the block): Account × Period block = 22,984 cells / 179 KB.
- ✅ **Sparse dimensions** (create the blocks): 11 dimensions (Subsidiary, Department, Scenario, Years, Version, Currency, ...).
- ⚠️ **Block density: Low (1.54% on Plan) — but expected** for a fully-aggregated BSO with 99% upper-level blocks. This does **not** mean flip a dimension. The savings path: clear stale scenarios first (Part 1), scope the AGG rules (Part 2), and only then optionally tune selected upper sparse members to dynamic calc (Section 3.5).

3.3 Real Essbase cube statistics (Activity Report)

Cube	Total blocks	Level-0	Upper-level	Density	Block size	Page file
Plan	337,653,497	3,645,406	334,008,091 (99%)	1.54%	179 KB	177,418 MB
Workforc	81,798,141	364,609	81,433,532 (100%)	0.3%	15 KB	77,263 MB
Details	13,462,360	90,667	13,371,693 (99%)	0.05%	158 KB	4,679 MB

- ✅ **Reading the upper-level share correctly:** ~99% upper-level blocks is **normal** for a fully-aggregated BSO — that alone is not a finding. The meaningful number is the **aggregation multiplier: 337,653,497 total ÷ 3,645,406 level-0 ≈ 93× stored aggregate blocks per input block**, and the fact that this multiplier applies to **all 47 scenarios and 12 years — including the stale ones nobody queries**.
- 💡 **Where the page-file savings actually come from:** (1) **clear the stale scenarios/years** (Section 1.2) — their stored aggregations go with them (up to the ~93× average; per-scenario aggregation varies, confirm in test); (2) **scope the AGG rules** so routine aggregations only touch active scenarios/years; (3) optionally, with **Hybrid enabled**, convert selected rarely-queried upper-level sparse members to dynamic calc (Section 3.5) — in waves with testing, NOT a blanket flip.

- ✕ **Outline warnings to clean:** Dynamic-calc L0 members WITHOUT a formula — Plan: Account[Total Cost of Goods Sold (FP&A)], Account[Deferred Expense (FP&A)], Account[Total Selling Expenses ...]; Workforc: Scenario[Fcst vs Act %]; Details: Account[Cumulative Translation Adjustment], Account[Cumulative Translation Adjustment-Elimination]; Dynamic-calc member with formula AND aggregating children — Plan: Account[Depreciation Accounts]; Gen-2 members not Never/Ignore with shared descendants — Subsidiary[ALT_SUBS] (all cubes), Plan: Account[GAAP_Other Current Liability].

3.4 Dimension design — change or keep? (Plan cube)

*Counts in this table come from the **Activity Report outline** (Plan cube only); Section 3.1's table counts come from the LCM across all cubes — small differences between the two sources are expected.*

#	Dimension	Current	Stored / declared	Recommendation
1	Account	Dense	1,352 / 2,841	✓ **Keep dense, as-is.** Stored upper accounts enlarge the BLOCK, not the block count — and Hybrid guidance says dense L1+ should NOT be dynamic. Leave dense alone; the lever is sparse (Section 3.5).
2	Period	Dense	17 / 37	✓ **Keep dense.** 17 of 37 members stored (BegBalance, TP1–TP12, key aggregates); rest dynamic — correct.
3	Class	Sparse	24 / 26	✓ **Keep sparse.** Appropriately sized; no change.
4	Subsidiary	Sparse	57 / 60	✓ **Keep sparse.** Appropriately sized; no change.
5	Location	Sparse	28 / 52	✓ **Keep sparse.** Appropriately sized; no change.
6	Department	Sparse	382 / 785	✓ **Keep sparse.** Correctly sparse — it creates blocks. 403 of 785 are non-stored (shared/dynamic).
7	Tracker	Sparse	60 / 77	✓ **Keep sparse.** Appropriately sized; no change.
8	Item	Sparse	36 / 40	✓ **Keep sparse.** Appropriately sized; no change.
9	Relationship	Sparse	46 / 47	✓ **Keep sparse.** Appropriately sized; no change.
10	Version	Sparse	2 / 3	✓ **Keep sparse.** Small — could be dense, but moving it would enlarge the block for little gain. Leave as-is.
11	Scenario	Sparse	60 / 92	💡 **Keep sparse — but CLEAN.** 60 stored scenarios is the bloat source (Section 1.2), not a design issue.
12	Currency	Sparse	10 / 12	✓ **Keep sparse.** Small — could be dense, but moving it would enlarge the block for little gain. Leave as-is.
13	Years	Sparse	19 / 20	✓ **Keep sparse.** Small — could be dense, but moving it would enlarge the block for little gain. Leave as-is.

Bottom line: no dimension should be flipped densesparse. The design is sound; the gains come from (a) **clearing stale scenarios** (Section 1.2), (b) **dynamic calc on the named sparse upper-level members in Section 3.5**, and (c) **outline order** (below).

3.5 Dynamic-calc candidates — tiered by conversion risk

Stored parents that could compute on the fly (dynamic calc)

29	43	14
SAFE (≤3 children)	MODERATE (4–10 children)	REVIEW (wide / formula)

Parents of small hierarchies (e.g. in Department) can stop being pre-calculated and stored — Hybrid computes them at query time with no noticeable cost. Details in Section 3.5.

Using the LCM's **parent-child data**, every **stored parent** in the Plan cube's sparse dimensions was tiered by how cheap it is to compute at query time (Hybrid):

- ✓ **SAFE** — ≤3 direct children, no formula: query-time cost is negligible. **Convert these first.**
- ! **MODERATE** — 4–10 children: fine in Hybrid; verify the hot reports that use them.
- ✕ **REVIEW** — >10 children or has a member formula: wide aggregation — test before converting; heavily-queried ones should stay stored.

Dimension	Stored parents	✅ SAFE	⚠️ MODERATE	❌ REVIEW	Example SAFE members (children)
Department	50	13	27	10	P_DEPT_18 (2), P_DEPT_21 (2), P_DEPT_24 (2)
Location	4	1	1	2	KPI_Location (3)
Tracker	6	2	4	0	MOD3Total (3), BS_DETAILS (3)
Subsidiary	15	10	5	0	TS (1), P_SUB_5 (2), ALT_SUBS (2)
Class	3	2	1	0	KPI_OrganicPlans (3), KPI_MigratedPlans (3)
Item	3	1	1	1	Inventory Item (1)
Relationship	5	0	4	1	—

29 SAFE parents can be converted with negligible risk (full member list with children counts in `dynamic-calc-candidates.csv`). Suggested wave plan: (1) all SAFE, (2) MODERATE in batches of ~10 with report-speed checks, (3) REVIEW only after confirming query usage. Department alone holds 50 of the stored parents — the hierarchy where this matters most.

*Correction note: an earlier level-based count suggested ~529 candidates; the parent-child analysis shows the true population is **86 stored parents** — the rest are alternate-hierarchy/shared structures that are not conversion targets.*

Outline order — today vs suggested

⚠️ The Activity Report flags **32 hourglass deviations** on the sparse dimensions. Best practice: dense first, then aggregating sparse from **smallest to largest stored members**, then non-aggregating sparse (Version/Scenario/Currency/Years) last. Counts = stored members per dimension; ❌ marks a dimension out of place today:

#	Today (current outline)	Suggested order
1	Account — dense	Account — dense
2	Period — dense	Period — dense
3	Class (24)	Class (24)
4	❌ Subsidiary (57)	Location (28)
5	Location (28)	Item (36)
6	❌ Department (382)	Relationship (46)
7	❌ Tracker (60)	Subsidiary (57)
8	Item (36)	Tracker (60)
9	Relationship (46)	Department (382)
10	Version (2)	Version — non-aggregating
11	Scenario (60)	Scenario — non-aggregating
12	Currency (10)	Currency — non-aggregating
13	Years (19)	Years — non-aggregating

💡 Out of place today: **Subsidiary (57), Department (382), Tracker (60)** — each sits before a smaller aggregating dimension. Re-ordering is metadata-only (no data change) but forces a full restructure: schedule it with the cleanup restructure in Section 1.4 so it costs nothing extra.

3.6 Data refresh from NetSuite (load window)

- ✅ **Actuals are loaded for closed months only.** Per the substitution variables, the last closed month is **TP5** and forecast starts at **TP6** (current month TP6, year FY26).
- ✅ The daily NetSuite sync (`ADMIN - Daily Sync AGG`) FIXes on **FY26, TP1:TP5, Actual** — i.e. it refreshes actuals for the closed periods of the current year, then aggregates.

- ⚠️ So **TP1–TP5 = NetSuite actuals; TP6+ = forecast/plan input.** Confirm with the client that no closed-period forecast data lingers past the load window (a common source of stale blocks).

3.7 AI / IPM Insights footprint

*In plain terms: Oracle's **IPM (Intelligent Performance Management)** is the built-in AI/ML layer of EPM/NSPB — **Auto Predict** (bulk statistical forecasts), **Prediction / Historical Insights** and **Anomaly Insights**. This section reads every IPM batch defined in the application straight from the LCM, so you can see what AI is configured, on which data, and with which model settings.*

- ✅ **Northwind uses IPM.** The LCM contains **5 Auto Predict batches** covering: Auto Predict, Prediction Insights, Anomaly Insights, Historical Insights.

Batch	Insight type(s)	Predicts (accounts)	History Future	Source Target (scn / version)	Model
AI Prediction (Expense)	Auto Predict	ILvl0Descendants(NFS_Expense), ILvl0Descendants(NFS_Cost of Sales)	24 mo 12 mo	NSP_Actual/NSP_Base NSP_Forecast/AI Prediction	ARIMA + seasonal + non-seasonal, fill missing, adjust outliers
AI Prediction (Expense) LRO	Auto Predict	ILvl0Descendants(NFS_Expense), ILvl0Descendants(NFS_Cost of Sales)	24 mo 12 mo	NSP_Forecast/NSP_Base NSP_Forecast/AI Prediction	ARIMA + seasonal + non-seasonal, fill missing
AI Prediction (Income)	Auto Predict	ILvl0Descendants(NFS_Income)	24 mo 12 mo	NSP_Actual/NSP_Base NSP_Forecast/AI Prediction	ARIMA + seasonal + non-seasonal, fill missing
AI Prediction (Income) LRO	Auto Predict	ILvl0Descendants(NFS_Income)	24 mo 12 mo	NSP_Forecast/NSP_Base NSP_Forecast/AI Prediction	ARIMA + seasonal + non-seasonal, fill missing
Revenue Insight	Auto Predict, Prediction Insights, Anomaly Insights, Historical Insights	ILvl0Descendants(NSP_Total Class), ILvl0Descendants(CAT_2)	36 mo 12 mo	4000/NSP_Actual 4000/Predicative Insight	ARIMA + seasonal + non-seasonal, fill missing, adjust outliers

- ✅ **Predictions are written to a dedicated `AI Prediction` version** (4 of 5 batches) — a clean, non-destructive design: the AI forecast never overwrites the planners' working numbers, it sits side-by-side for comparison.
- ✅ **"Revenue Insight" run the full Insights suite** (Auto Predict + Prediction + Historical + Anomaly) — the richest configuration: forecast, accuracy back-test and anomaly detection on the same series.
- ⚠️ **5 of 5 batches have `savePrediction = false`** — they compute the insight but do **not** persist results to the cube. Confirm whether that is intentional (preview/ad-hoc only) or whether the forecasts are meant to be saved and consumed in forms/reports.
- 💡 **All batches are univariate (each series predicted from its own history).** If drivers exist (e.g. headcount salary, volume revenue), a **Multivariate** Auto Predict can lift accuracy — a low-effort suggested change to pilot on the top revenue/expense lines.
- 💡 **Operationalize it (suggested change to validate):** if these are run manually, schedule the saved batches via **Application Jobs Schedule Jobs** (or an EPM Automate `runAutoPredict` step) so the AI forecast refreshes each cycle, and surface the `AI Prediction` version next to Forecast on the key revenue/expense forms so planners actually see and use it.

*All items above are **suggested changes** to review with the Northwind team — the LCM shows what is configured, not how often each batch is actually run or how its output is consumed.*

Part 4 — Suggested changes & next steps

4.1 Suggested changes (prioritized)

💡 *marks each **suggested change**. Everything below is a proposal to review with the Northwind team and validate in a test environment before applying — nothing has been implemented.*

1. 💡 **Fix the runaway CConv_Plan calc (235:00 min)** — it runs **single-threaded today**. Add `SET CALCPARALLEL 11;` (or FIXPARALLEL), scope the FIX to active years and run incrementally. Biggest single calc-time win. (Sections 2.1–2.3.)
2. 💡 **Clear/archive the 36 stale scenarios** (29.5% of input blocks) + FY20–FY23 — their stored aggregations go with them (estimated impact in Section 1.6). Then **scope the AGG rules to active scenarios/years** so the bloat doesn't regrow. **Take an LCM snapshot first.**
3. 💡 **Remove empty blocks and stored zeros** (Section 1.4) — clear-zeros custom calc + `CLEARBLOCK EMPTY` / explicit restructure after a snapshot; low-risk size win to validate in test.
4. 💡 **Convert thin stored parents to dynamic calc (optional, after #2)** — start with the **29 SAFE parents (≤3 children) named in Section 3.5** (mostly Department/Subsidiary), then MODERATE in batches of ~10 with report-speed checks. Skip REVIEW members used heavily in reports.
5. 💡 **Resolve outline warnings** (Section 3.3) and **fix alias gaps** (Account).
6. 💡 **Re-run the Activity Report + this analysis after cleanup** to confirm the block-count, page-file and calc-time drops.

4.2 Further analysis available (optional next steps)

Beyond this report, with the same data (or a short follow-up pull) BPC can also deliver:

- 💡 **Density by scenario** — find near-empty scenarios (cheap deletes) vs dense ones.
- 💡 **Per-account block contribution** — which accounts drive the most cells, candidates for dynamic calc.
- 💡 **Cache & config** — index cache 3,112 MB vs 8,679 MB index file; data cache 500 MB. _Caches are Oracle-managed in EPM Cloud_ — informational only; the lever is reducing data/blocks, not manual cache tuning.
- 💡 **Outline order / hourglass** optimization for faster calcs (32 sparse deviations today).
- 💡 **Calc parallelism audit** — which slow calcs run without Calc/Fix Parallel and could.
- 💡 **Restructure analysis** — the 10:12-min implicit refreshes; reduce restructure frequency/cost.
- 💡 **User adoption & Smart View compliance** — 0.3% UI requests over 2s, 99.93% availability; flag users on old Smart View versions.
- 💡 **Ongoing monitoring** — re-run this analysis monthly to catch scenario re-growth.

4.3 Estimated implementation effort & benefits

*All figures below are **conservative estimates** for one experienced EPM consultant, assuming the standard safe path: clone production to a test environment, apply each change there, validate with the client's key users, then promote to production in a planned window. Actual effort depends on client availability for validation — these are planning numbers to size the engagement, not a quote.*

#	Task	What it involves	Est. hours	Benefit
1	Test environment & baseline	Clone production to test, LCM snapshot, record baseline stats (block counts, page file, calc timings)	2	Safe sandbox; before/after proof for every change
2	Clear 36 stale scenarios	Confirm list with client, snapshot, clear scenario data in test, verify key forms/reports	4	Removes 29.5% of input blocks **and their stored aggregations** — the largest single space win
3	Archive FY20–FY22 to the Rpt (ASO) cube	Map/move historical actuals to the existing reporting cube, clear from Plan, point historical reports at Rpt	5	History stays queryable; BSO Plan cube stops aggregating years nobody plans on
4	Remove empty blocks	`CLEARBLOCK EMPTY` + explicit restructure, validate counts	2	~3,116,132 likely #Missing-only blocks dropped; faster restructures and backups
5	Clear-zeros calc	Build the custom calc (Section 1.4), test on the active scenarios, add to the post-load chain	2	1,580,715 zero cells stop being stored, aggregated and currency-converted

#	Task	What it involves	Est. hours	Benefit
6	Rework CConv_Plan	Parallelize, fix @CALCMODE, move years into the FIX, split incremental vs full-history version, benchmark (Section 2.3)	6	The 3.9 h single-threaded run becomes a scheduled, parallel, scoped calc — the biggest calc-time win
7	Parallelize remaining slow rules	Add CALCPARALLEL/FIXPARALLEL + tighter FIX to the other top calcs (Section 2.1), benchmark each	4	Shorter nightly window; users stop launching multi-hour interactive calcs
8	Dynamic-calc wave 1	Convert the 29 SAFE parents (Section 3.5), re-test the hot reports	3	Fewer stored upper blocks to aggregate; no report risk at this tier
9	Outline hygiene	Re-order sparse dims (Section 3.5), resolve outline warnings, fill alias gaps — share the restructure with task 4	3	Faster calcs from correct hourglass order; cleaner member display in reports
10	Full regression validation in test	Key forms, reports, month-end sequence end-to-end with client users; sign-off	5	Confidence that nothing user-facing changed before touching production
11	Production deployment	Migration window: snapshot, apply validated changes, re-run Activity Report + this analysis to confirm the drops	3	Measured before/after: page file, block counts, calc times

Total: ≈ 39 hours (roughly 5 consulting days; calendar span typically 2–3 weeks because validation windows depend on the client's close calendar). Tasks 1–5 alone (~15 h) deliver most of the **space** benefit; tasks 6–7 (~10 h) deliver most of the **calc-time** benefit — the program can be phased that way if needed.

Expected outcome if the full program is applied (estimates to confirm against the re-run Activity Report): Plan page file from **173 GB to roughly 120 GB**, the slowest calc from **3.9 h to well under 1 h**, a shorter nightly batch, and faster restructures/backups across the board.

Numbers are exact counts from the level-0 export (529,274 blocks, 6,251,076 lines parsed) and the Activity Report. Items labeled "estimated" (Section 1.6) are proportional estimates to validate in test.

Best practices per Oracle EPM Cloud / Hybrid BSO guidance: Design Best Practices (F96806), "Optimize BSO Cubes", and the CALCPARALLEL/FIXPARALLEL & CREATENONMISSINGBLK references in the Essbase Calculation guide.